Rapport Labo 2

Kenan Augsburger

08 November 2025

Contents

1. Code analysis

In this section, we'll go over the global constants and each function in the order they are called to explain what they do and how/where they are used.

> **Note**
>
> The explanation is mainly given as comments in the code accompanied with the addition of typing whenever it makes sense

1.1. Global constants

At the start of the scripts, we have multiple constants defined which are the parameters for the secp256k1

```python
# --- secp256k1 parameters ---
P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
A = 0
B = 7
Gx = 55066263022277343669578718895168534326250603453777594175500187360389116729240
Gy = 32670510020758816978083085130507043184471273380659243275938904335757337482424
N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

En comparant avec les paramètres que l'on trouve en ligne[1] on peut confirmer que ces paramètres sont correctes.

1.2. `main`

This is the entry point of the program.

1.2.1. `load_or_generate_keys`

This function is called at the start of `main` with this snippet

```python
    priv, pub = load_or_generate_keys()
```

This is the only time this function is called.

```python
def load_or_generate_keys() -> tuple[int, tuple[int, int]]:
    """This function loads an existing pair of private/public keys or generates,
    saves and return a new pair.
    """

    # Files containing the private and public keys
    priv_file = "ecdsa_private.key"
    pub_file = "ecdsa_public.key"
```

[1] https://neuromancer.sk/std/secg/secp256k1 °

```
9
10      if os.path.exists(priv_file) and os.path.exists(pub_file):
11          # If both files exist, proceed to extract the private and public keys.
12          with open(priv_file, "r") as f:
13              # The private key is converted from a base16 string to an int.
14              # NOTE: The fact that we strip what we read shouldn't matter
                unless
15              # the key isn't stored properly
16              priv = int(f.read().strip(), 16)
17          with open(pub_file, "r") as f:
18              # The public key is read as a JSON object with the keys x and y
19              # containing integers in base64
20              pub_data = json.load(f)
21              pub = (int(pub_data["x"], 16), int(pub_data["y"], 16))
22          print("Loaded existing keypair from disk.")
23      else:
24          print("No keypair found — generating new one...")
25          # If any of the files do not exist we generate a new pair of keys
26          priv, pub = generate_keypair()
27          with open(priv_file, "w") as f:
28              # The private key is simply written properly in hex format
29              f.write(hex(priv))
30          with open(pub_file, "w") as f:
31              # The public key is saved correctly in JSON
32              json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
33          print("New keypair generated and saved.")
34      return priv, pub
```

In this function we call `generate_keypair` to generate a pair of keys.

1.2.1.1. `generate_keypair`

```python
def generate_keypair() -> tuple[int, tuple[int, int]]:
    """Generate a pair of keys"""
    # Take a random value between 1 and N (N not included)
    priv = random.randrange(1, N)
    # Multiply the private key with G to get the pulic key
    pub = scalar_mult(priv, (Gx, Gy))
    return priv, pub
```

We take a random number between 1 and N for the private key

$$a \in \{1, ..., N-1\} \tag{1}$$

And for the public key, we call `scalar_mult`

1.2.1.1.1. `scalar_mult`

```python
def scalar_mult(k: int, point: tuple[int, int] | None):
    """Multiply a point on the curve by the scalar value k"""
```

```
    # Check that the point is on the curve otherwise the multiplication isn't
    # possible.
    assert is_on_curve(point)
    if k % N == 0 or point is None:
        # If k is perfectly divisible by N or now point was given, the result
        is None.
        # NOTE: If k is perfectly divisible by N, the result will be a None,
        # this check saves time
        return None
    if k < 0:
        # If k is negative, we simply multiply both inputs by -1 which is
        # simple to do and allows the next part of the algorithm to work.
        return scalar_mult(-k, (point[0], (-point[1]) % P))
    result = None
    addend = point
    # To calculate the result, we add point k times, effectively multiplying
    # the point by k
    while k:
        if k & 1:
            result = point_add(result, addend)
        addend = point_add(addend, addend)
        k >>= 1
    return result
```

This function should work. As a sanity check we can check with sage that the result is as expected.

```sage
E = EllipticCurve(GF(P), [A,B])
G = E(Gx,Gy)

for i in range(100):
     k = random.randrange(1,N)
     assert(scalar_mult(k, (Gx,Gy)) == (k * G).xy())

for i in range(100):
     k = -random.randrange(1,N)
     assert(scalar_mult(k, (Gx,Gy)) == (k * G).xy())
```

The only difference would be with the case of $k = 0$ since when done mathematically the result is $\mathcal{O}$ but since the program uses points with x and y coordinates it cannot represent this value. The choice of using `None` is valid but not that explicit. (Does every `None` represent $\mathcal{O}$ or is it an invalid value?)

1.2.1.1.1.1. `is_on_curve`

```python
def is_on_curve(point: tuple[int, int]):
    """Check if a point is on the Elliptic curve"""
    if point is None:
```

```
        return True
    x, y = point
    # Weistrass curve y^2 = x^3 + ax + b
    # NOTE: This is mathematically sound
    return (y * y - (x * x * x + A * x + B)) % P == 0
```

Given $k = (x, y)$ and $x, y \in \mathbb{Z}$, the function properly applies the Weistrass curve.

$$\begin{aligned} y^2 &= x^3 + ax + b \pmod{n} \\ y^2 - (x^3 + ax + b) &= 0 \pmod{n} \end{aligned} \tag{2}$$

2. Errors

Tâche 1

For each error:

- Show the snippet
- Explain what the problem is and the implications. What an attacker can do and what could go wrong.
- Explain in simple terms why this should be fixed
- Give a fix

2.1. `random.randrange`

This function is used twice in the code.

```python
def generate_keypair() -> tuple[int, tuple[int, int]]:
    """Generate a pair of keys"""
    priv = random.randrange(1, N)
    pub = scalar_mult(priv, (Gx, Gy))
    return priv, pub
```

```
def sign_message(priv, message):
    z = sha256(message)
    while True:
        k = random.randrange(0, N)
        R = scalar_mult(k, (Gx, Gy))
        r = R[0] % N
        if r == 0:
            continue
        k_inv = inv_mod(k, N)
        s = (k_inv * (z + r * priv)) % N
        if s == 0:
            continue
        return (r, s)
```

The documentation[2] for the module and its fonction starts with a warning to not use it for security and cryptographic purposes. This kind of warning shouldn't be ignored since it means that the prngs it provides are either not proved to be cryptographically secure or worse, proved to be attackable.

Furthermore, the range can be a problem independently of how good the distribution is. When multiplying a point with a scalar value, if the value is equal to zero or a multiple of N the result will be $\mathcal{O}$ which can be problematic.

For the first usage, the range gives us a value priv $\in \{1, \ldots, N-1\}$. This means that we shouldn't get a `None` but just to be safe we should have an assertion before returning our keys.

[2] https://docs.python.org/3/library/random.html

For the second usage, we have $k \in \{0, ..., N-1\}$ which means that R can be `None` which will raise an exception when accessing `R[0]`. This means that that there is a $\frac{1}{N}$ chance of having an issue when trying to sign a message.

We have two problems here.

2.1.1. Bad range $\{0, ..., N-1\}$

The problem here is that there is a chance of raising an exception and the implications vary depending on how exceptions are handled by the caller as well as which code paths lead to this function being executed.

The caller shouldn't expect `sign_message` to fail when given valid parameters. Depending on how they handle exceptions, the program could end up revealing informations which could be exploited or end up in an unrecoverable state.

If an attacker can trigger this function a lot, it could be used to leak secrets or for a denial of service attack.

If the exception results in an unrecoverable state such as the program exiting There is a serious risk of service degradation proportional to the usage.

In simple terms, the risk is that the service becomes unavailable or worse, that secrets could be leaked and used by bad actors to falsify prescriptions.

The simple fix is to change the range to $\{1, ..., N-1\}$. (Using an assertion as a sanity check and also allows the typing system to narrow after the multiplication)

```python
def sign_message(priv, message):
    z = sha256(message)
    while True:
        k = random.randrange(1, N)
        R = scalar_mult(k, (Gx, Gy))
        assert R
        r = R[0] % N
        if r == 0:
            continue
        k_inv = inv_mod(k, N)
        s = (k_inv * (z + r * priv)) % N
        if s == 0:
            continue
        return (r, s)
```

As stated previously, the range used in `generate_keypair` is correct but we should still have an assertion after the multiplication since it doesn't cost us much and having $\mathcal{O}$ as a public key would be catastrophic. (since $\forall k \in \mathbb{Z}, k\mathcal{O} = \mathcal{O}$)

```python
def generate_keypair() -> tuple[int, Point]:
    priv = random.randrange(1, N)
    pub = scalar_mult(priv, (Gx, Gy))
```

```
4       assert pub
5       return priv, pub
```

2.1.2. Usage of `random.randrange`

Tâche 2

snippet

Tâche 3

implications

Tâche 4

attacker

Tâche 5

what could go wrong

Tâche 6

simple terms why fix

Tâche 7

fix

2.2. `load_or_generate_keys`

2.2.1. Blind loading

```python
1   if os.path.exists(priv_file) and os.path.exists(pub_file):
2       # If both files exist, proceed to extract the private and public keys.
3       with open(priv_file, "r") as f:
4           # The private key is converted from a base16 string to an int.
5           # NOTE: The fact that we strip what we read shouldn't matter unless
6           # the key isn't stored properly
7           priv = int(f.read().strip(), 16)
8       with open(pub_file, "r") as f:
9           # The public key is read as a JSON object with the keys x and y
10          # containing integers in base64
11          pub_data = json.load(f)
```

```
12          pub = (int(pub_data["x"], 16), int(pub_data["y"], 16))
13      print("Loaded existing keypair from disk.")
14      # WARN: if both files exist, their value is taken as ground truth. There
15      #   is no verification that the private key is in [1;N[ and that the
16      #   public key is the result of priv * G
```

When loading the pair of keys from disk, there are no checks made to validate it. The problem is that to be able to properly sign and verify the signature of a message you need the private key a to respect $1 \leq a < N$ and the public key A should be $A = aG$.

If $a = 0$, the corresponding public key $A = \mathcal{O}$ which would be unusable and raise an exception whenever accessing either the x or y components of the point.

If $a = N$, then $A = \mathcal{O}$ since N is the order of the group. Same consequences.

Any other value of a will be treated as $a \bmod N$ so as long as $a \bmod N \neq 0$ the key should be valid but there shouldn't be any reason why we should have longer keys since it won't provide any benefits.

If $A \neq aG$, the signature verification won't work.

If A isn't on the curve, the program will stop after an assertion in `scalar_mult`.

If an attacker can modify the keys then you've got bigger problems than the program crashing from invalid keys.

One way this could go wrong is if there is a partial recovery, or maybe one of the keys was missing and the remaining key wasn't writable during the creation of a new pair. In those instances, the pair wouldn't match and lead to the program crashing.

This should be fixed because an invalid pair of keys will make the program unusable.

The simple fix would be to check that the keys are valid right after loading them and raise an exception to be handled otherwise. To make it easier to handle down the line, it also makes sense to split `load_or_generate_keys` into two distinct functions

```python
def compute_public_key(priv: PrivKey) -> PubKey:
    """Compute the public key of a private key"""
    pub = scalar_mult(priv, (Gx, Gy))
    assert pub is not None
    return pub


def is_valid_private_key(key: PrivKey) -> bool:
    """Check if a private key is valid with the curve"""
    return 1 ⩽ key < N


def is_valid_public_key(pub: PubKey, priv: PrivKey | None) -> bool:
    """Check if a public key is valid
```

```
    If None is specified for the private key, this function only checks
    if the
    key is on the curve
    """
    return is_on_curve(pub) if priv is None else pub ==
    compute_public_key(priv)


class InvalidPrivateKeyError(Exception):
    pass


class InvalidPublicKeyError(Exception):
    pass


def load_priv_key(path: str) -> PrivKey:
    """Load a private key from disk and check its validity
    Raises:
        FileNotFoundError: If the path doesn't lead to an existing file
        InvalidPrivateKeyError: If the key is invalid
    """
    if not os.path.exists(path):
        raise FileNotFoundError()
    with open(path, "r") as f:
        priv = int(f.read().strip(), 16)
        if not is_valid_private_key(priv):
            raise InvalidPrivateKeyError()
        return priv


def load_pub_key(path: str, priv_key: PrivKey | None) -> PubKey:
    """Load a public key from the disk and check its validity.
    Raises:
        FileNotFoundError: If the path doesn't lead to an existing file
        InvalidPublicKeyError: If the public key isn't the public key of the
            provided private key. If None is given for the private key,
            The only
            check is whether the public key is a point on the curve.
    """
    if not os.path.exists(path):
        raise FileNotFoundError()
    with open(path, "r") as f:
        # The public key is read as a JSON object with the keys x and y
        # containing integers in base64
        pub_data = json.load(f)
        pub = (int(pub_data["x"], 16), int(pub_data["y"], 16))
        if not is_valid_public_key(pub, priv_key):
```

```
            raise InvalidPublicKeyError()
        return pub


def generate_and_save_keys(priv_path: str, pub_path: str) -> tuple[PrivKey, PubKey]:
    if os.path.exists(priv_path):
        # NOTE: only keeping one backup since I don't want to bother with numbering
        os.replace(priv_path, priv_path + ".bak")
        print(
            f"existing private key found during generation of keys, backing up to {priv_path}.bak"
        )

    if os.path.exists(pub_path):
        # NOTE: only keeping one backup since I don't want to bother with numbering
        os.replace(pub_path, pub_path + ".bak")
        print(
            f"existing public key found during generation of keys, backing up to {pub_path}.bak"
        )

    priv, pub = generate_keypair()
    # WARN: The public and private keys are both simply saved to disk
    #   without any specifications when it comes to permissions. (on linux,
    #   resulted in 644 permissions)
    with open(priv_path, "w") as f:
        # The private key is simply written properly in hex format
        f.write(hex(priv))
    with open(pub_path, "w") as f:
        # The public key is saved correctly in JSON
        json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
    print("New keypair generated and saved.")
    return (priv, pub)


def generate_and_save_public_key(priv: PrivKey, pub_path: str) -> PubKey:
    if os.path.exists(pub_path):
        # NOTE: only keeping one backup since I don't want to bother with numbering
        os.replace(pub_path, pub_path + ".bak")
        print(
            f"existing public key found during generation of keys, backing up to {pub_path}.bak"
        )

```

```
    pub = compute_public_key(priv)
    with open(pub_path, "w") as f:
        # The public key is saved correctly in JSON
        json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
    return pub


def load_or_generate_keys() -> tuple[PrivKey, PubKey]:
    """This function loads an existing pair of private/public keys or
    generates,
    saves and return a new pair.
    """

    # Files containing the private and public keys
    priv_file = "ecdsa_private.key"
    pub_file = "ecdsa_public.key"

    try:
        priv = load_priv_key(priv_file)
    except (InvalidPrivateKeyError, FileNotFoundError) as err:
        print(
            "Your private key is invalid."
            if isinstance(err, InvalidPrivateKeyError)
            else "Missing private key"
        )
        choice = input(
            "Would you like to generate a new pair of keys? Existing keys
            will be backed up to [key].bak y/N: "
        )
        if not choice == "y":
            raise err
        return generate_and_save_keys(priv_file, pub_file)

    try:
        pub = load_pub_key(pub_file, priv)
    except (InvalidPublicKeyError, FileNotFoundError) as err:
        print(
            "Your public key is invalid."
            if isinstance(err, InvalidPublicKeyError)
            else "Missing public key"
        )
        choice = input("Would you like to generate the public key? y/N: ")
        if not choice == "y":
            raise err
        return (priv, generate_and_save_public_key(priv, pub_file))

    return (priv, pub)
```

> **Note**
>
> This rewrite of the `load_or_generate_keys` is a bit messy but it handles the issue of loading invalid keys as well as the issue of irreversibly losing keys without warning. (Described in more details in the next section)

2.2.2. Destructive storage of private key

```python
else:
    print("No keypair found — generating new one...")
    # If any of the files do not exist we generate a new pair of keys
    priv, pub = generate_keypair()
    with open(priv_file, "w") as f:
        # The private key is simply written properly in hex format
        f.write(hex(priv))
```

If only private key exists, the function will overwrite it with a new private key. This is destructive since we can always compute the public key from the private key but we cannot recover the private key once overwritten.

This means that if for some reasons, the public key isn't found we will get a new pair of keys. We will need to distribute our new public key to anyone who needs to verify our signatures and if we can't our old public key anywhere, the validity of past records becomes unverifiable.

This most likely isn't an attack vector. If an attacker manages to delete the public key on the server they probably already compromised the server.

This should be fixed because it's a threat to the trust of previously signed records.

The simple fix to this issue is to prompt the user and ask whether they want to overwrite the private key or compute the public key and save it.

This is implemented in the fix of the previous section.

2.2.3. Insecure permission management of private key

```python
else:
    print("No keypair found — generating new one...")
    # If any of the files do not exist we generate a new pair of keys
    priv, pub = generate_keypair()
    with open(priv_file, "w") as f:
        # The private key is simply written properly in hex format
        f.write(hex(priv))
    with open(pub_file, "w") as f:
        # The public key is saved correctly in JSON
        json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
    print("New keypair generated and saved.")
    # WARN: The public and private keys are both simply saved to disk
    #   without any specifications when it comes to permissions. (on linux,
```

```
14      #   resulted in 644 permissions)
```

When either a public key or private key file is missing, a new pair of keys is generated and saved to disk using the built-in open function in `'w'` mode.

The keys will have the default permissions (determined by the OS). On Linux, it often means `0644` which is an issue since we don't want any other user to be able to read the private key.

In the event of an attacker gaining access to the host running the program, they can extract the private key without any privilege escalation. This would allow them to falsify records. (Breaks authenticity and integrity)

Effectively, this would mean that the attacker could create a fake prescription and the pharmacy would treat it as valid. For doctors, this would allow the attacker to create fake symptoms for a patient which in turn would result in more work for the doctors to check and effectively destroy the efficiency brought by the system.

This issue should be fixed because it could cause serious legal troubles since it involves medical prescriptions. Furthermore, it also risks the loss of trust in the system.

```python
priv, pub = generate_keypair()
(priv_mask, pub_mask) = (0o600, 0o644)
flags = os.O_WRONLY | os.O_CREAT | os.O_TRUNC

# Get open the file through the os api to enforce permissions on the files
priv_fd = os.open(priv_path, flags, priv_mask)
os.fchmod(priv_fd, priv_mask)

with os.fdopen(priv_fd, "w") as f:
    # The private key is simply written properly in hex format
    f.write(hex(priv))

pub_fd = os.open(pub_path, flags, pub_mask)
os.fchmod(pub_fd, pub_mask)
with os.fdopen(pub_fd, "w") as f:
    # The public key is saved correctly in JSON
    json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
print("New keypair generated and saved.")
```

2.3. Domain issues (We don't know which private key was used to sign)

Tâche 8

fill, concerns about the system only caring about « now » and no concerns of key rotations

Tâche 9

snippet

Tâche 10

issue

Tâche 11

implications

Tâche 12

attacker

Tâche 13

what could go wrong

Tâche 14

simple terms why fix

Tâche 15

fix

3. Conclusion

Other than cryptographic issues, there are also a lot of problems with the structure of the code. If the goal is to provide a library to use within another program, it should be clearly structured in modules and ensure that the library is used correctly.

We would need classes to ensure that public and private keys are valid on instantiation as well as clear exceptions to handle errors.